Machine-Learning-Based Predictive Maintenance: Remaining-Useful-Life Prediction on the NASA C-MAPSS Turbofan Engine Degradation Dataset

Moksh Mehta
Brown University
moksh_mehta@brown.edu

Independent Study, Spring 2026

Abstract

Condition-monitoring sensors produce dense multivariate time series that can be used to predict machine failure before it occurs, replacing scheduled maintenance with condition-based intervention. This study develops an end-to-end machine-learning pipeline for remaining-useful-life (RUL) prediction and evaluates it on the NASA C-MAPSS turbofan engine degradation dataset (Saxena and Goebel, 2008), the standard academic benchmark for this task. The pipeline comprises operating-regime detection, regime-aware standardisation, rolling statistical features and a piecewise-linear RUL target. Five tabular baselines (mean predictor, ridge regression, random forest, XGBoost, LightGBM) and a two-layer LSTM are trained and evaluated on all four C-MAPSS subsets, and the best model is served through a Flask REST API that returns a numeric RUL prediction together with a categorical maintenance recommendation. On the held-out test sets, the best models achieve RMSE of **15.27**, **14.74**, **14.47** and **15.50** cycles on FD001–FD004 respectively, competitive with results reported in the prognostics literature. The study also documents a subtle data-leakage failure mode: a normalised cycle-counter feature that is perfectly correlated with RUL on training engines, which all run to failure, but uninformative on test engines, which are truncated mid-life. This feature inflated tree-model RMSE from about 17 to 85 cycles, and it would not be detected by conventional validation on training engines.

1 Introduction

Industrial assets such as jet engines, pumps, bearings and gearboxes are typically maintained under one of three regimes: *run-to-failure* (lowest capital cost, highest cost of unplanned downtime), *scheduled/preventive* (replacing parts at fixed intervals regardless of their actual condition), or *condition-based/predictive* (intervening only when the data indicate that the asset is approaching failure) (Lei et al., 2018). Predictive maintenance (PdM) promises a lower total cost of ownership, but it requires a model that maps streams of sensor data to an actionable estimate of the asset's future state.

This study addresses that problem on the NASA C-MAPSS Turbofan Engine Degradation Simulation Dataset (Saxena and Goebel, 2008). The dataset has been used in hundreds of studies since its release, which provides a clear reference for performance on each of its four subsets. It also poses the central modelling questions of predictive maintenance: how to handle multiple operating regimes, how to define a sensible regression target while the asset is healthy, how to evaluate fairly

given the asymmetric costs of early and late predictions, and how to translate a regression output into an operational decision.

Contributions. This report makes four contributions:

1. A reusable, documented Python pipeline (data loading, regime-aware feature engineering, sequence construction, model training and a REST API), released as a self-contained repository.
2. Benchmarks of five classical regressors and an LSTM on all four C-MAPSS subsets, reported in RMSE and MAE.
3. A case study of a feature-leakage bug that inflates test RMSE fivefold and is not detected by ordinary hold-out validation on training engines.
4. A deployment prototype (a Flask REST API and a Streamlit dashboard) that maps numerical RUL predictions onto categorical maintenance actions.

Section 2 defines the problem and the evaluation metrics, Section 3 describes the dataset and exploratory analysis, and Section 4 details the features and models. Section 5 reports test-set performance and the leakage case study, Section 6 describes the deployment prototype, and Section 7 discusses limitations and future work.

2 Problem statement and evaluation

2.1 Definitions

For each asset (here, a turbofan engine) a multivariate time series $\{x_t\}_{t=1}^{T}$ is observed, where each $x_t \in \mathbb{R}^d$ collects the values of d sensor channels at operational cycle t. In the training data each asset is observed until failure, so $T = T_{\text{fail}}$. At any cycle $t \leq T_{\text{fail}}$, the *remaining useful life* (RUL) of the asset is

$$\text{RUL}(t) \;=\; T_{\text{fail}} - t. \tag{1}$$

The supervised task is to predict $\widehat{\text{RUL}}(t)$ from the history $x_{1:t}$.

2.2 Evaluation metrics

RMSE. The root-mean-square error between predicted and true RUL on the test set; symmetric and directly interpretable in cycles.

MAE. The mean absolute error; less sensitive than RMSE to the few engines whose dynamics differ strongly from the rest of the fleet.

PHM08 score. The asymmetric scoring function introduced for the 2008 PHM Society competition (Saxena and Goebel, 2008),

$$S \;=\; \sum_{i=1}^{N} s_i, \quad s_i = \begin{cases} \exp(-d_i/13) - 1 & d_i < 0 \\ \exp(\,d_i/10) - 1 & d_i \geq 0 \end{cases}, \quad d_i = \widehat{\text{RUL}}_i - \text{RUL}_i. \tag{2}$$

Because the exponent for over-estimates (10) is smaller than that for under-estimates (13), a prediction that *over-estimates* RUL by 30 cycles is penalised much more heavily than one that under-estimates it by the same amount. This encodes the operational fact that *a late prediction is worse than an early one*: an engineer told that an engine has 50 cycles left risks a failure if the true value is 20. The score is used here to motivate the decision thresholds of the deployment prototype (Section 6); quantitative results are reported in RMSE and MAE.

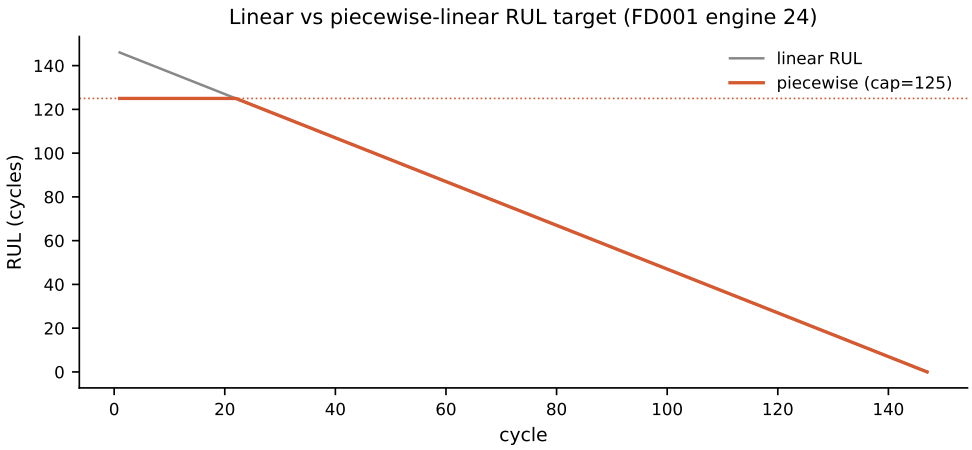


Figure 1: Linear vs. piecewise-linear RUL target for one FD001 engine. Capping at $R_{\max} = 125$ replaces the early life of the engine with a single "healthy" value, focusing learning on the degradation phase.

2.3 The piecewise-linear RUL target

A purely linear RUL target (Eq. 1) asks the model to do something that is not possible early in an engine's life: distinguish a new engine that will run for 300 cycles from one that will run for 250. Following Heimes (2008), the training target is capped at an upper bound of 125 cycles, the standard choice in the literature:

$$\mathrm{RUL}^{\dagger}(t) \;=\; \min\bigl(\mathrm{RUL}(t), R_{\max}\bigr), \qquad R_{\max} = 125. \tag{3}$$

In the healthy phase, the model is therefore only asked to predict "≥ 125", and learning is concentrated on the degradation phase (Figure 1).

3 Dataset and exploratory analysis

3.1 The C-MAPSS dataset

C-MAPSS (Frederick et al., 2007) (Commercial Modular Aero-Propulsion System Simulation) is a high-fidelity, physics-based simulator of a large commercial turbofan engine developed at NASA Glenn Research Center. Saxena and Goebel (2008) used it to generate run-to-failure trajectories for a fleet of nominally identical engines, each with a small random degree of initial wear and manufacturing variation, and released four subsets of increasing difficulty (Table 1). Each record contains three operational settings (altitude proxy, Mach proxy and throttle resolver angle) and 21 sensor measurements (temperatures, pressures, fan speeds, fuel flow and bleed enthalpy) for one operational cycle of one engine.

3.2 Engine lifetimes

Figure 2 shows the distribution of engine lifetimes in the training sets. The mean lifetime is 200–250 cycles, but the right tail extends beyond 500 cycles. A model trained on the raw RUL would therefore need to span a target range of roughly 500 cycles, although most of the informative variation occurs in the final 100 or so, which motivates the piecewise-linear target of Eq. 3.

Table 1: Summary of the four C-MAPSS subsets. "Train cycles" is the total number of (engine, cycle) records available for training. "Mean length" is the average number of recorded cycles per engine: full run-to-failure lifetimes for training engines, and truncated trajectories for test engines.

Subset	Op. conditions	Fault modes	Engines		Train cycles	Mean length (cycles)	
			train	test		train	test
FD001	1 (sea level)	1 (HPC)	100	100	20 631	206.3	131.0
FD002	6	1 (HPC)	260	259	53 759	206.8	131.2
FD003	1 (sea level)	2 (HPC + Fan)	100	100	24 720	247.2	166.0
FD004	6	2 (HPC + Fan)	249	248	61 249	246.0	166.2

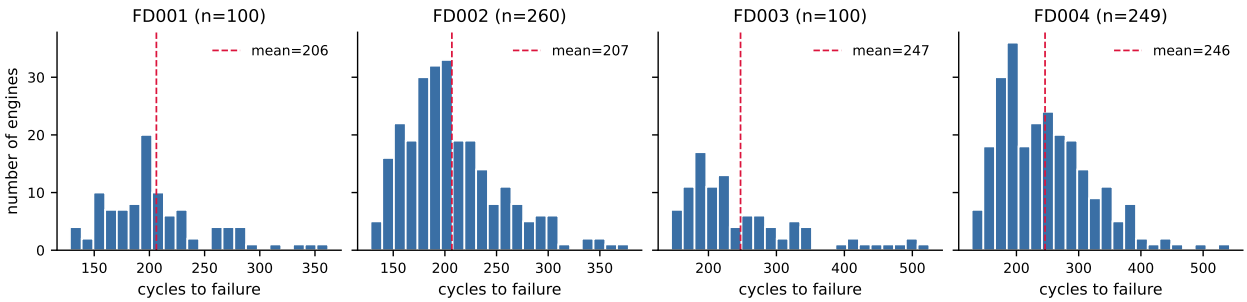


Figure 2: Distribution of run-to-failure lengths in the training set of each subset. The dashed red line marks the mean.

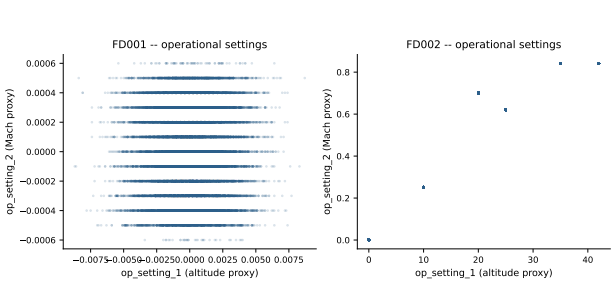
3.3 Operating conditions

FD001 and FD003 are recorded at a single operating condition (sea-level cruise), so their operational-setting columns are essentially constant. FD002 and FD004 sweep through six discrete conditions corresponding to different altitude, Mach and throttle combinations. Figure 3a plots `op_setting_1` against `op_setting_2` for FD001 (a single point cloud) and FD002 (six clusters). K-means with $k = 6$ recovers the six clusters in FD002 and FD004 exactly (Figure 3b), and this clustering is used for regime-aware normalisation.

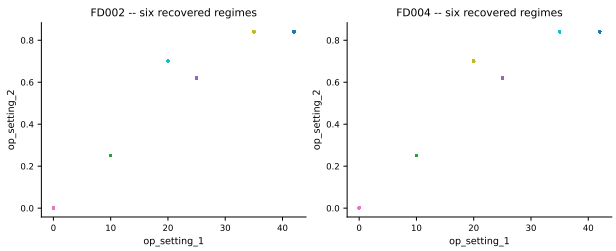
3.4 Sensor signals

Seven of the 21 sensor channels (`s1, s5, s6, s10, s16, s18, s19`) carry little or no variance in FD001 and FD003, because the quantities they measure are essentially constant under sea-level cruise (Figure 4). These channels are excluded from all subsequent modelling. The remaining sensors show a clear degradation signature over an engine's lifetime (Figure 5): values drift monotonically, some upward and some downward, with stochastic cycle-to-cycle variation around the trend.

Per-sensor correlation with RUL. Figure 6 ranks each sensor by its Pearson correlation with the uncapped RUL in each subset. On the single-condition subsets (FD001, FD003), about ten sensors are clearly correlated ($|r| > 0.3$). On the multi-condition subsets (FD002, FD004), raw correlations are uniformly weak because switching between operating conditions dominates the cycle-to-cycle variation, which motivates the regime-aware normalisation described next.



(a) Raw scatter of the first two operational settings.



(b) K-means with $k = 6$ recovers the six discrete regimes.

Figure 3: Operational regimes in C-MAPSS. The multi-condition subsets FD002 and FD004 contain six well-separated flight regimes.

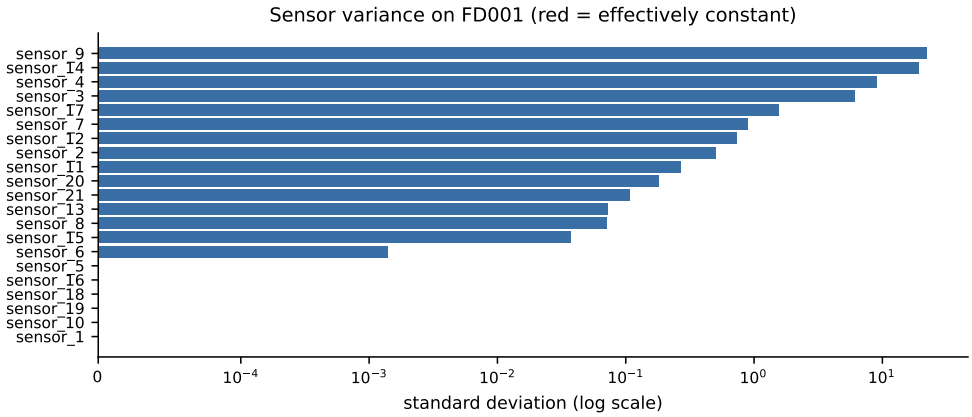


Figure 4: Per-channel standard deviation in FD001 (log scale). The seven channels with the smallest standard deviation are effectively constant and are excluded from the feature set.

4 Methods

4.1 Regime-aware normalisation

For the single-condition subsets, each retained sensor channel is z-scored using the mean and standard deviation of the training set. For the six-condition subsets, all training rows are first clustered with K-means on the three operating settings ($k = 6$), a separate `StandardScaler` is fitted to each cluster, and at inference each new row is routed to the scaler of its cluster. This standard technique (Ramasso and Saxena, 2014) removes the dominant operating-condition variance and exposes the underlying degradation signal (Figure 7).

4.2 Engineered features for tabular models

For each standardised sensor, the following statistics are computed per engine over windows $w \in \{5, 20\}$:

- the rolling mean $\bar{x}_w(t)$,
- the rolling standard deviation $\sigma_w(t)$,
- the rolling least-squares slope $\hat{\beta}_w(t)$ of the channel against time, computed in closed form from cumulative sums so that the cost is $\mathcal{O}(N)$ rather than $\mathcal{O}(Nw)$.

Together with the standardised values themselves, this gives $14 \times 7 = 98$ features per cycle. The

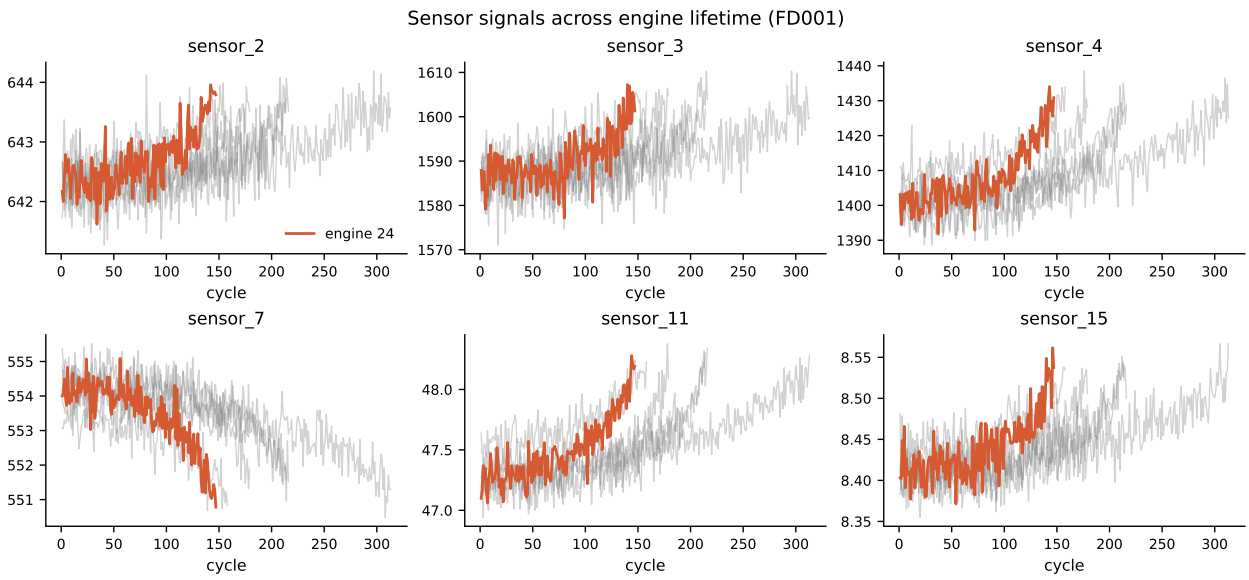


Figure 5: Six representative sensor traces over the lifetime of one FD001 engine (red) and several others (grey). The drifts in the final 100–150 cycles of each trace are the degradation signature to be predicted.

tempting feature `cycle_norm` = `cycle`/ max(`cycle`) is deliberately excluded; Section 5.2 explains why.

4.3 Sequence dataset for the recurrent model

The LSTM uses a sliding window of length $L = 30$ cycles. Each training cycle t yields one example $X_t = (x_{t-29}, \ldots, x_t)$, zero-padded at the front for $t < 30$, with target $y_t = \min(\mathrm{RUL}(t), 125)$. Each test engine contributes only its final L recorded cycles, paired with the supplied ground-truth RUL. The sequences contain only the 14 standardised sensors (plus the regime label for FD002 and FD004), without the engineered rolling statistics, so that the LSTM learns its own temporal aggregation.

4.4 Models

Six models are trained and evaluated on each subset.

Mean. Predicts the training-set mean RUL for every test engine; the trivial baseline that any useful model must beat.

Ridge. Linear regression with an L_2 penalty ($\alpha = 1$). It fits in under a second per subset and indicates how much of the problem is linear in the engineered features.

Random Forest. 80 trees, maximum depth 12, minimum 10 samples per leaf. To limit computation, this model is evaluated only on the two smaller subsets (FD001, FD003).

XGBoost. 400 boosting rounds, maximum depth 6, learning rate 0.05, L_1/L_2 regularisation of 0.1/1.0, `tree_method = hist`.

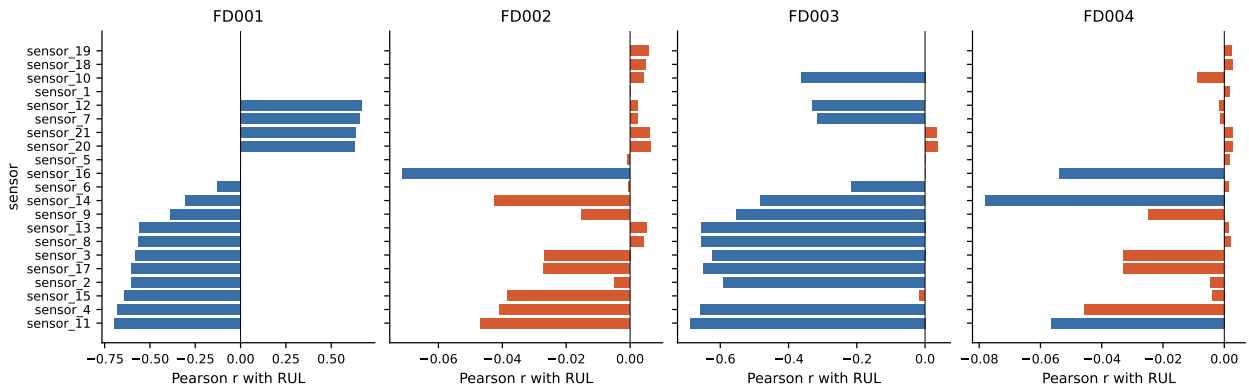


Figure 6: Per-sensor Pearson correlation with raw RUL in each subset. On the multi-condition subsets (FD002, FD004), raw correlations are uniformly weak because the operating regime dominates the variation; normalising within each regime is required before the correlations become meaningful.

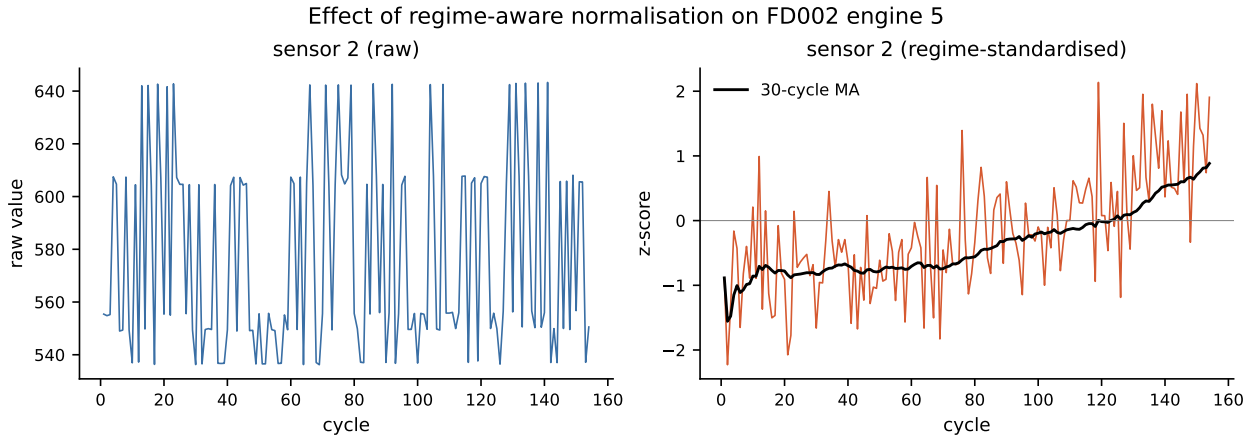


Figure 7: Raw vs. regime-standardised sensor 2 trace for one FD002 engine. The raw signal (left) is dominated by jumps between flight regimes; the regime-standardised signal (right, with a 30-cycle moving average in black) reveals the gradual degradation trend.

LightGBM. 500 rounds, 63 leaves, learning rate 0.05; otherwise configured as XGBoost.

LSTM. Two stacked LSTM layers (hidden size 64, dropout 0.2), followed by a 32-unit ReLU layer and a linear output, implemented in PyTorch (Paszke et al., 2019) with approximately 56k parameters. The model is trained with Adam ($\eta = 10^{-3}$, weight decay 10^{-5}), MSE loss, gradient clipping at norm 1.0, a batch size of 512 and a `ReduceLROnPlateau` scheduler. Fifteen percent of the training engines, split by engine rather than by cycle, form the validation set used for early stopping.

All experiments use a fixed random seed (0).

5 Results

5.1 Test-set performance

Table 2 reports test-set RMSE and MAE for every model and subset. The LSTM achieves the lowest RMSE on three of the four subsets (FD001, FD002, FD004) and is within 0.5 cycles of

Table 2: Test-set performance on the four C-MAPSS subsets. RMSE and MAE are in cycles (lower is better). The best value in each column is in bold; dashes mark models not evaluated on that subset.

	Test RMSE (cycles)				Test MAE (cycles)			
Model	FD001	FD002	FD003	FD004	FD001	FD002	FD003	FD004
Mean	41.94	44.93	43.70	45.57	34.83	38.14	36.44	38.06
Ridge	17.77	18.04	18.16	21.36	14.11	14.43	15.01	17.41
RandomForest	17.05	—	15.32	—	12.17	—	10.47	—
XGBoost	15.66	15.14	15.01	16.63	11.53	10.81	10.31	12.08
LightGBM	15.93	14.91	**14.47**	16.50	11.46	10.56	**9.69**	11.95
LSTM	**15.27**	**14.74**	14.97	**15.50**	**10.83**	**10.53**	9.98	**11.14**

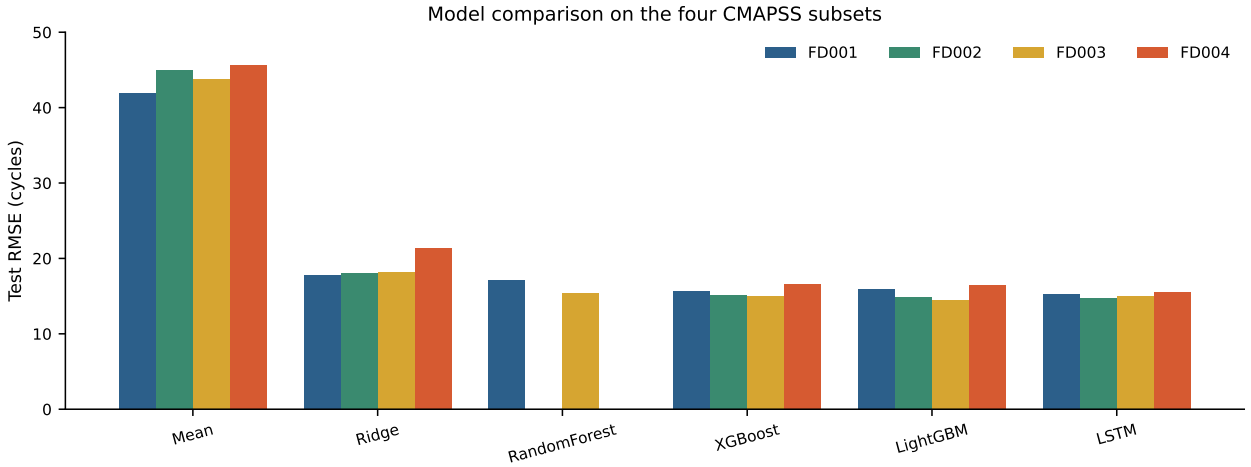


Figure 8: Test RMSE for every model and subset. The mean predictor is included as a reference; every learned model improves on it by a factor of 2–3. The LSTM is the strongest model on three of the four subsets.

LightGBM on FD003. Both gradient-boosted tree models come within 0.3–0.7 cycles of the LSTM on every subset, at a much lower training cost (10–25 seconds compared with several minutes). Ridge regression closes more than half of the gap between the mean predictor and the best model without any tuning, indicating that simple statistics of the sensor trends capture much of the prognostic signal. Figure 8 summarises these results, and Figure 9 shows the LSTM training curves: all four runs converge within 10–25 epochs without signs of overfitting, consistent with the small parameter count and the engine-level validation split.

Calibration. Figure 10 compares predicted and true RUL for the LSTM. For test engines whose true RUL exceeds the cap, predictions saturate near 125 cycles, as intended by the piecewise-linear target. In the operationally important regime (RUL < 60), predictions cluster around the diagonal with a scatter of about ± 10 cycles, consistent with the reported MAE.

Per-engine error. Figure 11 shows the signed prediction error for the 100 FD001 test engines, sorted by true RUL. Engines far from failure are predicted slightly too low on average, an operationally benign error that leads to early inspection rather than missed maintenance. Engines close

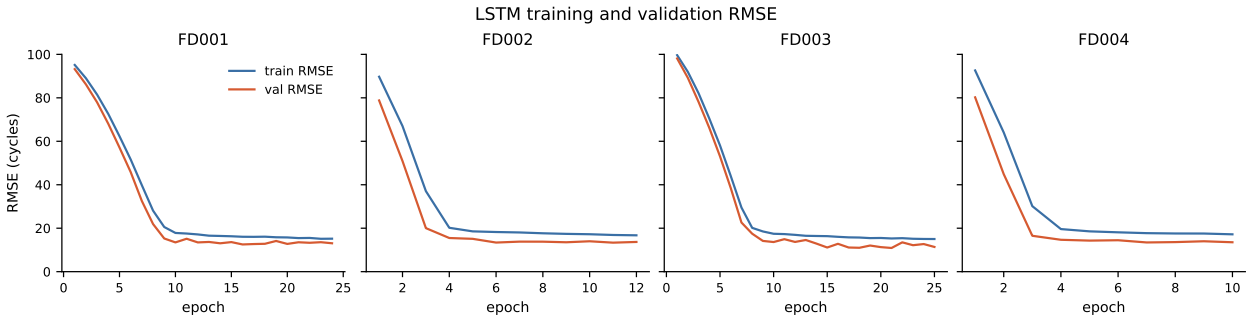


Figure 9: LSTM training (blue) and validation (red) RMSE per epoch on each subset.

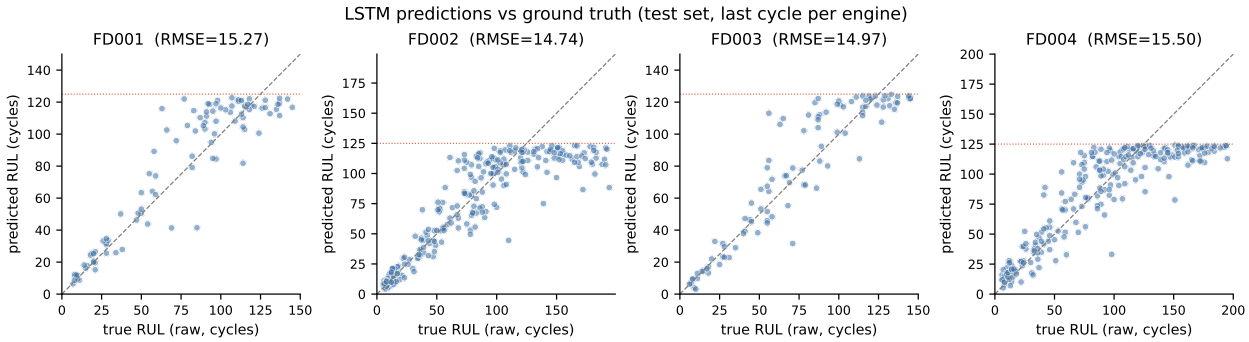


Figure 10: Predicted vs. true RUL for the LSTM on each subset (test set, last cycle per engine). The dashed grey line indicates perfect prediction; the dotted red line marks the training cap $R_{\max} = 125$. Saturation at the cap for healthy engines is by design (Eq. 3).

to failure show small residuals of both signs, and no large late predictions (for example, a true RUL of 5 predicted as 80) occur.

Feature importance. Figure 12 ranks the 20 most important features of the LightGBM model on FD001. Rolling statistics of sensors 14, 9, 17, 3, 13 and 15 dominate the ranking; these are the channels with the largest variance and the strongest monotone drift over an engine's lifetime (Figure 5). All 20 features use the long window ($w = 20$) rather than the short one ($w = 5$), consistent with RUL evolving over tens of cycles rather than single cycles. Rolling trend features (e.g. `sensor_12_trend_w20`, `sensor_14_trend_w20`) also rank highly, indicating that the recent slope of a sensor is a more direct indicator of degradation than its instantaneous value.

5.2 A data-leakage failure mode

An early version of the tree models reached an RMSE of 84.5 on FD001, worse than the mean predictor (41.9), while ridge regression on the same features achieved 17.8. The discrepancy was traced to a single feature that appeared useful:

```
df["cycle_norm"] = df.groupby("unit")["cycle"]\
                    .transform(lambda c: c / c.max())
```

Because every training engine runs to failure, `cycle_norm == 1.0` coincides exactly with RUL $= 0$ in the training data. Boosted trees, which learn such piecewise-constant relationships easily, adopt the rule "if `cycle_norm` is close to 1, predict RUL ≈ 0". Test engines, however, are truncated *before* failure: their last recorded cycle also has `cycle_norm == 1.0`, but the true RUL at that point

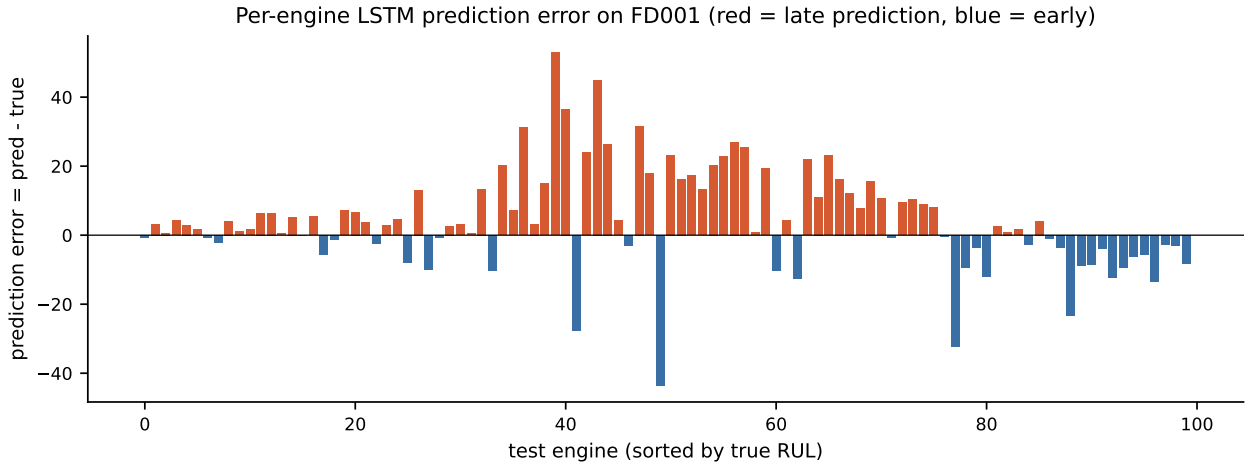


Figure 11: Signed prediction error for every FD001 test engine, sorted by true RUL (lowest RUL on the right). Red bars are late predictions (over-estimates); blue bars are early predictions (under-estimates).

ranges from 7 to 145 cycles. The models therefore predicted near-zero RUL for healthy engines, and the RMSE increased sharply. Ridge regression was much less affected, because it assigned the feature only a modest linear weight among dozens of others. Removing the feature reduced the FD001 RMSE of random forest from 84.5 to 17.1, XGBoost from 69 to 15.7 and LightGBM from 76 to 15.9.

Lesson. With run-to-failure training data and right-censored test data, any feature normalised by the length of a training trajectory is suspect. This bug is not detected by a validation split that respects engine boundaries within the training set, because the validation engines also run to failure; it is detected only by validating against truncated trajectories. Deployed systems should therefore reserve a small set of *truncated* validation trajectories, even when full-life data are available.

6 Deployment prototype

6.1 REST API

The trained LightGBM model is served through a Flask (Ronacher, 2010) REST endpoint (`src/api.py`). The API accepts JSON of the form

```
{ "cycles": [[op1, op2, op3, s1, ..., s21],   // cycle 1
             [op1, op2, op3, s1, ..., s21],   // cycle 2
             ...                              // in chronological order
           ] }
```

and returns

```
{ "model": "LightGBM-FD001",
  "subset": "FD001",
  "n_cycles_received": 31,
  "predicted_RUL_cycles": 123.47,
  "warning_level": "OK",
  "recommended_action":
       "Continue normal operation. Re-evaluate after next cycle." }
```

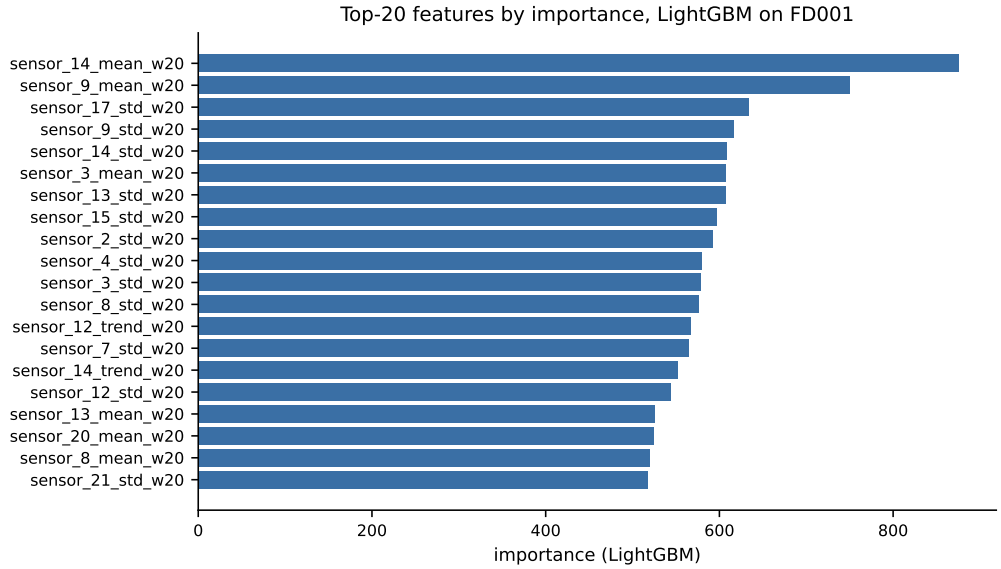


Figure 12: Top-20 features by importance for the LightGBM model on FD001. The ranking is dominated by rolling means, standard deviations and trends computed over the longer ($w = 20$) window.

Predicted RUL is mapped to a maintenance action using deliberately coarse thresholds:

Predicted RUL	Warning level	Recommended action
> 80 cycles	OK	Continue normal operation.
31–80	WATCH	Add to elevated-attention list; review weekly.
11–30	PLAN_MAINTENANCE	Schedule maintenance within 10 cycles.
≤ 10	CRITICAL	Take asset offline at next opportunity.

In practice, these thresholds would be tuned by the asset owner against the relative costs of early and late intervention, as captured by the PHM08 score (Eq. 2).

6.2 Dashboard

A Streamlit (Inc., 2019) dashboard (`src/dashboard.py`) allows an engineer to select a held-out test engine, inspect its sensor traces and the model's prediction, and read the recommended action. Figure 13 shows the dashboard layout for FD001 engine 90, whose true RUL is 28 cycles: the model predicts 26.7 cycles and issues the PLAN_MAINTENANCE recommendation.

7 Limitations and future work

Simulated data. C-MAPSS is generated by a physics-based simulator rather than collected from engines in service. Its noise model is idealised; sensor faults and missing data are absent; and all channels are perfectly synchronised. Real monitoring data would require additional preprocessing for outlier detection, multi-rate sensor alignment and gap filling.

Frequency-domain features. The C-MAPSS sensors measure slowly varying thermodynamic quantities (temperatures, pressures and fan speeds). Many industrial monitoring systems instead record high-frequency vibration signals. The modelling backbone (regime-aware standardisation,

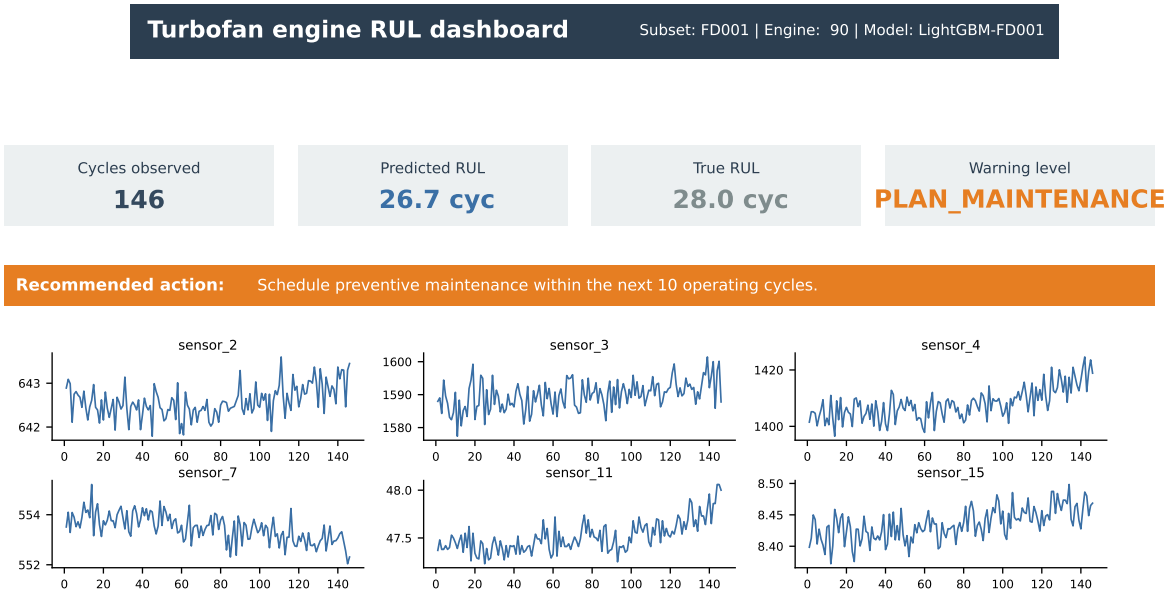


Figure 13: Dashboard layout for FD001 engine 90 (true RUL = 28 cycles). The model correctly recommends planned maintenance.

rolling features and the LSTM) would transfer unchanged, but feature extraction would gain a spectral component, such as band-pass energy ratios around bearing characteristic frequencies and the kurtosis and crest factor of the envelope spectrum (Randall and Surveillance, 2011).

Limited fault modes. FD003 and FD004 contain only two fault modes. Real fleets exhibit a longer tail of failure modes (seal wear, blade-tip rub, bearing spalling, loss of lubrication), each with its own degradation signature. A deployed system would require either separate models per mode or a multi-task model that predicts both the failure mode and the time to failure.

Cost-sensitive thresholds. The 80/30/10-cycle thresholds of the prototype are placeholders. In deployment, they should be optimised against the actual costs of unplanned downtime and scheduled inspection, for example by using the PHM08 score with an asymmetry tuned to that cost ratio.

Uncertainty quantification. The current models produce point estimates. Maintenance decisions would benefit from a *predictive distribution*, so that recommendations can be tied to a stated confidence (e.g. "95% probability that RUL $\leq$ 30"). Quantile regression (Koenker, 2005) and ensemble or Monte-Carlo dropout methods (Gal and Ghahramani, 2016) are natural extensions.

8 Conclusion

This study presents a complete, reproducible predictive-maintenance pipeline for the NASA C-MAPSS dataset, comprising regime detection, regime-aware standardisation, rolling-statistic features, a piecewise-linear RUL target, five classical baselines, a PyTorch LSTM and a Flask/Streamlit deployment prototype that translates numerical RUL into maintenance actions. The best test-set RMSE lies between 14.5 and 15.5 cycles on every subset, competitive with published results. The study also documents a leakage failure mode, a normalised cycle-count feature that inflated

tree-model RMSE fivefold and is not detected by standard validation on run-to-failure data, and recommends that any system trained on such data reserve right-censored validation trajectories to catch this class of error.

Acknowledgements

The author thanks Prof. Serdar Kadioglu for advising this independent study, and the NASA Ames Prognostics Center of Excellence for releasing the C-MAPSS dataset under terms that permit academic use. The dataset is acknowledged as requested by its custodians: "A. Saxena and K. Goebel (2008). 'Turbofan Engine Degradation Simulation Data Set', NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA".

References

D. K. Frederick, J. A. DeCastro, and J. S. Litt. User's guide for the commercial modular aero-propulsion system simulation (c-mapss). *NASA/TM*, 2007-215026, 2007.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *arXiv preprint arXiv:1506.02142*, 2016.

Felix O. Heimes. Recurrent neural networks for remaining useful life estimation. In *Proceedings of the International Conference on Prognostics and Health Management*, pages 227–234, 2008.

Streamlit Inc. Streamlit: The fastest way to build and share data apps, 2019. URL `https://streamlit.io/`.

Roger Koenker. *Quantile Regression*. Cambridge University Press, 2005.

Yaguo Lei, Bo Yang, Xian Jiang, Fuqing Jiao, Naipeng Li, and Torsten Sommerfeld. Applications of structural health monitoring to machine prognostics. *Mechanical Systems and Signal Processing*, 99:1–14, 2018.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019.

Emmanuel Ramasso and Abhinav Saxena. Investigating the effect of operating condition on machine health monitoring in a turbofan engine. *Journal of Engineering for Gas Turbines and Power*, 136 (8):081202, 2014.

Robert B. Randall and Vibration Surveillance. Vibration-based condition monitoring: Industrial, aerospace and automotive applications. *John Wiley & Sons*, 2011.

Armin Ronacher. Flask: A lightweight wsgi web application framework, 2010. URL `https://flask.palletsprojects.com/`.

Abhinav Saxena and Kai Goebel. Turbofan engine degradation simulation data set. *NASA Prognostics Data Repository*, 2008.

A Reproducibility

All experiments were run on CPU with Python 3.12, using `numpy 2.4`, `pandas 3.0`, `scikit-learn 1.8`, `xgboost 3.2`, `lightgbm 4.6`, `pytorch 2.12` and `scipy 1.17`, with all random seeds set to 0. Approximate end-to-end run times on a single CPU:

- data download and parsing: ≤ 10 s;
- exploratory analysis and figures: ≈ 30 s;
- all tabular baselines on all four subsets: ≈ 4 min;
- LSTM on FD001: ≈ 3 min (24 epochs, early-stopped);
- LSTM on FD002: ≈ 4 min (12 epochs);
- LSTM on FD003: ≈ 3 min (25 epochs);
- LSTM on FD004: ≈ 4.5 min (10 epochs).

A GPU would reduce LSTM training time by roughly an order of magnitude but was not required.

B Repository contents

The code repository contains the following files, all documented in `README.md`: `src/data_loader.py`, `src/features.py`, `src/eda.py`, `src/baselines.py`, `src/lstm_model.py`, `src/result_figures.py`, `src/api.py`, `src/dashboard.py` and `src/dashboard_mockup.py`, together with the trained models `models/lightgbm_FD001.joblib`, `models/xgboost_FD001.joblib`, `models/xgboost_FD003.joblib` and `models/lstm_FD001.pt` through `models/lstm_FD004.pt`.